
Randomized Advantage Transformation (RAT): Computing Natural Policy Gradients via Direct Backpropagation

Mingfei Sun¹

Abstract

Natural policy gradients improve optimization by accounting for the geometry of distribution space, but their practical use is limited by the cost of estimating and inverting the Fisher matrix. We present Randomized Advantage Transformation (RAT), a method for estimating Tikhonov-regularized natural policy gradients via direct backpropagation. By applying the Woodbury formula, we reformulate the regularized natural policy gradients as vanilla policy gradients with a transformed advantage. RAT computes this transformation efficiently via randomized block Kaczmarz iterations on on-policy mini-batches, avoiding explicit Fisher construction, conjugate-gradient solvers, and architecture-specific approximations. We provide convergence guarantees for RAT and demonstrate empirically that it matches or exceeds established natural-gradient methods across continuous and visual control benchmarks, while remaining simple to implement and compatible with various architectures.

1. Introduction

Natural policy gradients are a foundational tool in deep Reinforcement Learning (RL), offering parameterization-invariant update directions (Bagnell & Schneider, 2003) by pre-conditioning policy gradients with the inverse Fisher matrix (Amari, 1998; Kakade, 2001). This geometric correction has been shown to significantly improve convergence properties (Agarwal et al., 2021), and underlies several influential RL algorithms, including Natural Actor-Critic (NAC) (Peters & Schaal, 2008), Trust Region Policy Optimization (TRPO) (Schulman et al., 2015), ACKTR (Wu et al., 2017), and connections to PPO-style updates (Schulman et al., 2017b; Hilton et al., 2022).

¹The University of Manchester, United Kingdom. [Code URL](#).
Correspondence to: <mingfei.sun@manchester.ac.uk>.

Despite their advantages, natural policy gradients are rarely used directly in large-scale deep reinforcement learning due to computational constraints. The Fisher matrix scales with the number of policy parameters, making explicit construction or inversion infeasible. To address this, prior work has largely followed two directions. Hessian-free approaches compute Fisher-vector products and solve the resulting linear systems using conjugate gradient methods, as in TRPO (Schulman et al., 2015). While effective, these methods introduce substantial computational overhead, require careful tuning of inner-loop solvers, and are difficult to apply in shared actor-critic architectures (Schulman et al., 2017b; Wu et al., 2017). Alternatively, structured approximations such as KFAC (Martens & Grosse, 2015) exploit layer-wise factorizations of the Fisher matrix, trading accuracy for efficiency but relying on architecture-dependent assumptions (Benzing, 2022) and nontrivial implementation optimizations (George et al., 2018).

In this work, we show that estimating natural policy gradients can be reduced to a simpler and more general procedure. Our starting point is the observation that Tikhonov-regularized natural policy gradients admit an equivalent least-squares formulation. By applying the Woodbury formula, we derive a representation in which the inverse Fisher matrix is absorbed into a transformation of the advantage function. Under this reformulation, the natural policy gradient takes the same form as a vanilla policy gradient, but with a modified advantage, see Figure 1 for an overview.

Building on this insight, we introduce Randomized Advantage Transformation (RAT), an algorithm that approximates the transformed advantage using randomized block Kaczmarz iterations (Needell & Tropp, 2014). RAT operates on small subsets of on-policy samples and iteratively refines an estimate of the regularized natural policy gradient. Each iteration requires only standard backpropagation through a surrogate loss, eliminating the need for explicit Fisher construction, Fisher-vector products, or architecture-specific curvature approximations.

We provide a convergence analysis showing that RAT converges linearly to the regularized natural policy gradient under standard assumptions on the function approximation and state-action coverage. We evaluate RAT on a range

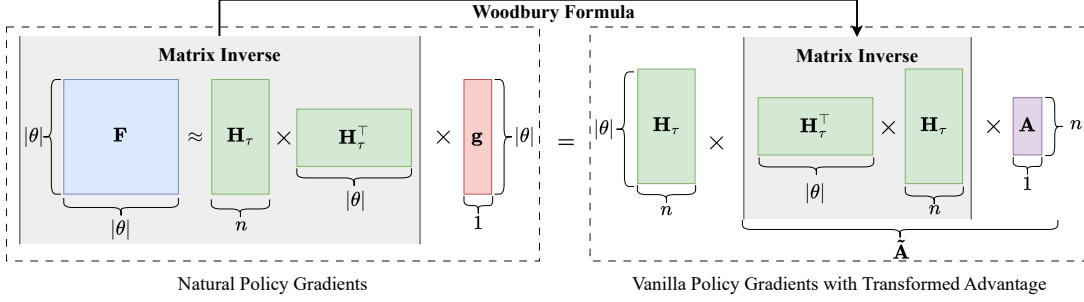


Figure 1. Fisher matrix $\mathbf{F} \in \mathbb{R}^{|\theta| \times |\theta|}$ estimated from samples τ (shown in green) is often ill-conditioned and hard to invert directly. Randomized Advantage Transformation (RAT) leverages Woodbury formula to replace the inversion of $\mathbf{F}$ to that of a sampled preconditioner of size $n \times n$ (shown in grey). The resulting inverse is absorbed into a randomized, block-wise transformation of the advantage function, yielding a surrogate objective whose first-order gradient via direct backpropagation approximates natural policy gradient.

of benchmark reinforcement learning tasks, including continuous control in MuJoCo (Brockman et al., 2016) and high-dimensional visual control in Procgen (Cobbe et al., 2020). Across these settings, RAT matches or outperforms established natural policy gradient methods while offering a simpler implementation and broader applicability, including support for shared actor-critic architectures. Our contributions are as follows:

- We derive a Woodbury-based reformulation of Tikhonov-regularized natural policy gradients as vanilla policy gradients with transformed advantages.
- We propose Randomized Advantage Transformation (RAT), an efficient algorithm based on randomized block Kaczmarz iterations that estimates NPG using standard backpropagation.
- We provide theoretical convergence guarantees and empirical evidence demonstrating the effectiveness of RAT on diverse benchmarks.

2. Related work

We review prior work on estimating natural policy gradients and on using the Woodbury formula to reduce the cost of Fisher inversion.

Estimating natural policy gradients. The central computational challenge in natural policy gradient methods is to *estimate or apply the inverse Fisher matrix* efficiently and robustly in the presence of sampling noise and function approximation. A widely adopted strategy avoids forming the Fisher explicitly and instead computes *Fisher Vector Products* (FVP), also known as Hessian-free methods (Martens, 2010; Pascanu & Bengio, 2014). The resulting linear systems are typically solved using iterative methods such as conjugate gradient (CG) algorithm. This approach underlies trust-region style reinforcement learning algorithms (Schulman et al., 2015) and many modern natural policy gradient implementations (Kuba et al., 2022; Sun et al., 2023). In

practice, however, achieving sufficient accuracy often requires many CG iterations at each training iteration, leading to substantial computational overhead. To address this issue, alternative approaches rely on *structured approximations* of the Fisher matrix, including diagonal (Liu et al., 2024), layer-wise block-diagonal (Martens & Grosse, 2015; George et al., 2018), or low-rank forms (Dangel et al., 2023; Yang et al., 2022). Among these, Kronecker-Factored Approximate Curvature (KFAC) has been widely used in reinforcement learning (Wu et al., 2017; Bae et al., 2022). KFAC approximates each layer’s Fisher matrix as a Kronecker product of two smaller matrices, relying on independence assumptions about the statistics of the gradients (Ba et al., 2017) or their eigen-structure (George et al., 2018). While effective, KFAC depends strongly on the gradient structure of the neural networks. In contrast, our method RAT is oblivious to the particular model architecture and estimates natural policy gradients through standard backpropagation. Another influential line of work estimates natural policy gradients via *natural actor-critic* (NAC) formulations (Peters & Schaal, 2008; Cayci & Eryilmaz, 2025). In these methods, the advantage function is represented in a *compatible* form (Sutton et al., 1999), ensuring that the natural policy gradient emerges exactly as the solution to a least-squares regression problem (Kakade, 2001; Schulman et al., 2017a). Our work is related in spirit, but differs in how the least-squares problem is constructed and solved.

Woodbury for inverting Fisher. The Woodbury formula provides an efficient way to compute matrix inverses and solve linear systems involving low-rank updates. Its use in natural gradients dates back to Amari et al. (2000), where the Fisher inverse can be stored explicitly and is estimated directly using the Sherman-Morrison lemma, a special case of the Woodbury formula. More recently, this idea has been extended to rank-1 approximations of natural policy gradients with neural parameterization (Huo et al., 2026). Woodbury formula has also been used to reduce the computational cost

of natural gradient optimization steps. For example, [Chen & Heyl \(2024\)](#) apply Woodbury-based updates to accelerate natural gradient methods, and related work develops a momentum scheme to further improve convergence, e.g., SRPING ([Goldshlager et al., 2024](#)). Only very recently has the full Woodbury formula been used to reformulate the *Tikhonov-regularized natural gradients*. For instance, [Wu et al. \(2024\)](#) employs the push-through identity to analyze per-sample loss reduction under natural gradient updates, while [Guzmán-Cordero et al. \(2025\)](#) uses Woodbury-based transformations to reduce the cost of Fisher inverse, for which convergence guarantees have been provided in [Goldshlager et al. \(2026\)](#). Our approach builds on these insights but differs in how the Woodbury reformulation is exploited. Instead of directly approximating the inverse Fisher, we use the Woodbury identity to transform the advantage function and then estimate the resulting natural policy gradient iteratively through standard backpropagation.

Position of RAT. RAT differs from prior natural gradient methods in that it neither relies on conjugate-gradient solvers nor on structured Fisher approximations. Instead, it leverages a Woodbury-based reformulation to shift curvature information into an advantage transformation, which is approximated via randomized linear solvers. This perspective allows RAT to remain architecture-agnostic, computationally efficient, and compatible with various architectures.

3. Preliminaries

Reinforcement learning formulation. We consider a standard reinforcement learning setup in which an agent interacts with an environment over discrete timesteps. At each timestep t , the agent observes a state $s_t \in \mathcal{S}$, selects an action $a_t \in \mathcal{A}$ and receives a scalar reward $r_t \in \mathbb{R}$. The agent’s behavior is defined by a stochastic policy π , which maps states to action distributions $\pi : \mathcal{S} \mapsto \Delta_{|\mathcal{A}|}$ (where $\Delta_d := \{\mathbf{x} \in \mathbb{R}_+^d : \sum_i x_i = 1\}$ denotes the probability simplex). The environment is modelled as a Markov Decision Process (MDP) $\{\mathcal{S}, \mathcal{A}, P, r, p_0\}$, where $P(\cdot|s, a)$ is the transition kernel, $r(s, a)$ is the reward function, and p_0 is the initial state distribution. The (discounted) return from time step t is defined as $R_t := \sum_{i=t}^T \gamma^{i-t} r(s_i, a_i)$, where $\gamma \in [0, 1)$ is the discount factor. The objective is to learn a policy that maximizes the expected return from the initial state distribution. The action-value function of a policy π is defined as $Q_\pi(s_t, a_t) := \mathbb{E}[R_t|s_t, a_t]$. The value function V_π and advantage function $A_\pi(s, a)$ are given by: $V_\pi(s_t) := \mathbb{E}_{a_t \sim \pi(\cdot|s_t)}[Q_\pi(s_t, a_t)]$, and $A_\pi(s, a) := Q_\pi(s, a) - V_\pi(s)$. We define the discounted state distribution as $d_\pi(s) := (1 - \gamma) \sum_{t=0}^{\infty} \gamma^t P(s_t = s|\pi, p_0)$, where the factor $(1 - \gamma)$ ensures normalization. With slight abuse of notation, we use $d_\pi(s, a)$ to mean $s \sim d_\pi(s), a \sim \pi(\cdot|s)$.

Vanilla and Natural Policy Gradients. We consider a parameterized policy $\pi(a|s; \theta)$. Let π_k denote the policy induced by parameters θ_k , and d_k denote the corresponding discounted state-action distribution. The vanilla policy gradient is given by ([Sutton et al., 1999](#)):

$$\nabla_{\theta}^{\text{PG}} J(\theta) := \mathbb{E}_{(s,a) \sim d_\pi} \left[\frac{\partial}{\partial \theta} \log \pi(a|s; \theta) A_\pi(s, a) \right].$$

When the parameter space has a non-Euclidean geometry, the vanilla gradient does not correspond to the direction of steepest ascent. To address this, [Amari \(1998\)](#) proposed the natural gradient, which accounts for the information geometry of the parameter manifold. In reinforcement learning, [Kakade \(2001\)](#) introduced the natural policy gradient based on the Fisher matrix

$$\mathbf{F}(\theta) := \mathbb{E}_{d_\pi(s,a)} \left[\frac{\partial}{\partial \theta} \log \pi(a|s; \theta) \frac{\partial}{\partial \theta^\top} \log \pi(a|s; \theta) \right].$$

The resulting natural policy gradient is defined as

$$\nabla_{\theta}^{\text{NPG}} J(\theta) := \mathbf{F}(\theta)^{-1} \nabla_{\theta}^{\text{PG}} J(\theta).$$

We focus on Kakade’s formulation of the natural policy gradient, which has been shown to be invariant to reparameterization ([Bagnell & Schneider, 2003](#)). Following [Kunstner et al. \(2019\)](#), we refer to the Fisher matrix estimated from finite samples as the *empirical Fisher*, and call the resulting gradients the *Empirical Natural Policy Gradients*.

Randomized block Kaczmarz method. The randomized block Kaczmarz method is an iterative algorithm for solving overdetermined least-squares problems of the form:

$$\min_{\mathbf{x}} \|\mathbf{A}\mathbf{x} - \mathbf{b}\|_2^2, \quad (1)$$

where $\mathbf{A} \in \mathbb{R}^{n \times d}$ and $\mathbf{b} \in \mathbb{R}^n$. Starting from an initial estimate $\mathbf{x}_0$, the method iteratively refines the solution by projecting onto the solution space of randomly selected row blocks of $\mathbf{A}$. Specifically, let $T = \{\tau_1, \tau_2, \dots, \tau_m\}$ be a partition of the rows of $\mathbf{A}$. At iteration j , a block $\tau_j \in T$ is sampled (typically uniformly at random), and the update is:

$$\mathbf{x}_j = \mathbf{x}_{j-1} + \mathbf{A}_{\tau_j}^\top (\mathbf{A}_{\tau_j} \mathbf{A}_{\tau_j}^\top)^{-1} (\mathbf{b}_{\tau_j} - \mathbf{A}_{\tau_j} \mathbf{x}_{j-1}).$$

Under standard assumptions, this randomized block scheme converges to the least-squares solution, often with favorable convergence properties compared to deterministic variants ([Needell & Tropp, 2014](#)).

4. Randomized Advantage Transformation

In this section, we first show that Tikhonov-regularized natural policy gradients (NPG) can be formulated as a regularized least-squares problem. We then apply the Woodbury formula to express the resulting NPG update in terms of a transformed advantage, reducing it to a vanilla policy gradient form. Finally, we introduce a randomized Kaczmarz iteration to efficiently solve the corresponding least squares.

4.1. Tikhonov Regularized NPG

As shown by [Kakade \(2001\)](#), the natural policy gradient can be obtained as the solution to a least squares problem. We follow the notation of [Schulman et al. \(2017a\)](#). Use n to denote the cardinality of state-action space, i.e., $n = |\mathcal{S}||\mathcal{A}|$, and p to denote the number of policy parameters, i.e., $p = |\theta|$. Let $\Sigma \in \mathbb{R}^{n \times n}$ be a diagonal matrix with diagonal entries $d_\pi(s)\pi(a|s)$, $H \in \mathbb{R}^{n \times p}$ be the matrix whose rows are $\frac{\partial}{\partial \theta} \log \pi(a|s; \theta)$, $y \in \mathbb{R}^n$ be the vector with entries $A_\pi(s, a)$. Under this notation, the vanilla PG and NPG can be written as:

$$\text{PG: } \nabla_\theta^{\text{PG}} J(\theta) := H^\top \Sigma y, \quad (2)$$

$$\text{NPG: } \nabla_\theta^{\text{NPG}} J(\theta) := (H^\top \Sigma H)^{-1} H^\top \Sigma y. \quad (3)$$

The NPG update coincides with the solution to the following least squares.

Proposition 1 ([Kakade \(2001\)](#)). *The minimizer of $\min_x \|y - Hx\|_\Sigma^2$ is given by $x^* = (H^\top \Sigma H)^{-1} H^\top \Sigma y$, provided the inverse exists.*

In practice, the Fisher matrix is estimated from a finite number of samples, (i.e., empirical Fisher), which can render it ill-conditioned or singular. To stabilize inversion, it is common to apply Tikhonov regularization by adding a damping term ([Schulman et al., 2015](#); [Martens & Grosse, 2015](#)).

$$\text{T-NPG: } \nabla_\theta^{\text{T-NPG}} J(\theta) := (\lambda I + H^\top \Sigma H)^{-1} H^\top \Sigma y, \quad (4)$$

where $\lambda > 0$ is a damping coefficient. This Tikhonov-regularized NPG (T-NPG) is equivalently the solution to the regularized least-squares:

$$\begin{aligned} &(\lambda I + H^\top \Sigma H)^{-1} H^\top \Sigma y \\ &= \arg \min_g \|y - Hg\|_\Sigma^2 + \lambda \|g\|_2^2 \end{aligned} \quad (5)$$

This formulation motivates our use of iterative least-squares solvers in the following section.

4.2. Woodbury Reformulation

Interestingly, the introduction of Tikhonov regularization enables the use of the Woodbury formula to transform the inverse ([Wu et al., 2024](#); [Guzmán-Cordero et al., 2025](#)). Applying the formula, $(I + UV)^{-1}U = U(I + VU)^{-1}$ for any conformable matrices U, V , twice yields

$$\nabla_\theta^{\text{T-NPG}} J(\theta) = (\lambda I_p + H^\top \Sigma H)^{-1} H^\top \Sigma y \quad (6)$$

$$= H^\top (\lambda I_n + \Sigma H H^\top)^{-1} \Sigma y \quad (7)$$

$$= H^\top \Sigma \boxed{(\lambda I_n + H H^\top \Sigma)^{-1} y}. \quad (8)$$

Compared to vanilla policy gradients $\nabla_\theta^{\text{PG}} J(\theta) = H^\top \Sigma y$, the only difference is the advantage term. Specifically, T-NPG can be written as $\nabla_\theta^{\text{T-NPG}} J(\theta) = H^\top \Sigma \tilde{y}$ where

$$\tilde{y} = (\lambda I_n + H H^\top \Sigma)^{-1} y. \quad (9)$$

Thus, the T-NPG corresponds to a vanilla policy gradient with a *transformed advantage function*,

$$\bar{A}_\pi(s, a) := [(\lambda I_n + H H^\top \Sigma)^{-1} y]_{(s,a)}. \quad (10)$$

Importantly, the matrix inversion in the above transformation does not depend on the number of parameters p since $\lambda I_n + H H^\top \Sigma$ is of size $n \times n$. This important property is the key to develop our method.

4.3. Randomized Kaczmarz Iteration

A major limitation of the above advantage transformation is that n is typically much larger than p , essentially in continuous state-action spaces, making the exact transformation infeasible. Since T-NPG naturally arises from a least-squares problem, we turn to randomized iterative solvers for linear systems ([Gower & Richtárik, 2015](#)) and propose to approximate the advantage transformation via randomization.

[Equation \(5\)](#) involves weighting by Σ , which is generally unknown. To bypass this, we replace this objective with an unweighted least squares problem constructed from on-policy samples. This corresponds to a Monte Carlo approximation in which state-action pairs are drawn i.i.d from $d_\pi(s)\pi(a|s)$, so that expectations over Σ are replaced by empirical averages. Specifically, sampling $(s, a) \sim d_\pi(s)\pi(a|s)$ yields

$$\mathbb{E}_\tau [\|y_\tau - H_\tau g\|_2^2] \propto \|y - Hg\|_\Sigma^2. \quad (11)$$

Under standard on-policy assumptions, the resulting estimator is unbiased in expectation, and the discrepancy introduced by ignoring Σ vanishes as the batch size increases.

To solve [Equation \(5\)](#), we adopt randomized block Kaczmarz method ([Needell & Tropp, 2014](#)). Let $\mathcal{D}_k$ denote on-policy samples at iteration k , partitioned into mini-batches $\{\tau_1, \tau_2, \dots, \tau_m\}$. Starting from an initial estimate g_0 , at iteration j , we select a batch τ_j , and perform the following:

$$g_j \leftarrow \arg \min_g \|y_{\tau_j} - H_{\tau_j} g\|_2^2 + \lambda \|g - g_{j-1}\|_2^2. \quad (12)$$

Here, H_{τ_j} denotes the rows of H indexed by τ_j . Note that this corresponds to a regularized block update in the randomized block Kaczmarz framework. In the classical (unregularized) formulation, each step projects the current iterate onto the solution space of the sampled linear system $H_{\tau_j} g = y_{\tau_j}$. However, enforcing this hard constraint directly can be unstable in our setting due to noise and potential rank deficiency of minibatches. Thus, we consider the regularized version, which can be viewed as a proximal update that balances fitting the current batch with staying close to the previous estimate. Importantly, as shown in [Goldshlager et al. \(2024\)](#), [Equation \(12\)](#) admits a closed-form

update

$$\mathbf{g}_j \leftarrow \mathbf{g}_{j-1} + \mathbf{H}_{\tau_j}^\top \underbrace{(\lambda \mathbf{I} + \mathbf{H}_{\tau_j} \mathbf{H}_{\tau_j}^\top)^{-1} (\mathbf{y}_{\tau_j} - \mathbf{H}_{\tau_j} \mathbf{g}_{j-1})}_{\text{Randomized Advantage Transformation}}. \quad (13)$$

The bracketed term performs an advantage transformation on the sampled data. We therefore refer to this method as *Randomized Advantage Transformation* (RAT), i.e.,

$$\tilde{A}_j(s, a) := \left[(\lambda \mathbf{I} + \mathbf{H}_{\tau_j} \mathbf{H}_{\tau_j}^\top)^{-1} (\mathbf{y}_{\tau_j} - \mathbf{H}_{\tau_j} \mathbf{g}_{j-1}) \right]_{(s,a)}. \quad (14)$$

With minibatch size B we have $\mathbf{H}_\tau \in \mathbb{R}^{B \times p}$ and $\mathbf{y}_\tau \in \mathbb{R}^B$. When $B \ll p$, the matrix inversion is only $B \times B$, in contrast to the original $n \times n$ matrix. The main computational cost arises from forming $\mathbf{H}_\tau \mathbf{H}_\tau^\top$, which scales as $\mathcal{O}(pB^2)$; this cost can be further reduced using Nyström Approximation (Gittens & Mahoney, 2013). Further, as pointed by Guzmán-Cordero et al. (2025), $\mathbf{H} \mathbf{H}^\top$ is the neural tangent kernel (Jacot et al., 2018), and thus can be approximated efficiently in various ways (Novak et al., 2022). $\mathbf{H}$ can be estimated efficiently using the per-sample gradients in PyTorch¹. The advantage can be computed via `torch.linalg.solve`: which directly solve linear system with matrix $(\lambda \mathbf{I} + \mathbf{H} \mathbf{H}^\top)$ and vector $\mathbf{y}$. It is faster and more numerically stable than explicitly computing the inverse.

The natural policy gradients can then be computed via direct backpropagation using the following PPO-like objective:

$$J_{\text{RAT}}(\boldsymbol{\theta}) := \mathbb{E}_{(s,a) \sim \mathcal{D}_k} \left[\frac{\pi(a|s; \boldsymbol{\theta})}{\pi_{\text{old}}(a|s)} \tilde{A}_j(s, a) \right], \quad (15)$$

where π_{old} is the behavior policy used to collect $\mathcal{D}_k$, and remains the same during the inner iterations.

Intuition Underlying RAT. RAT can be viewed as an efficient method for constructing a compatible approximation of advantage function by solving linear system $\mathbf{H} \mathbf{x} = \mathbf{y}$ with Tikhonov regularization. At each iteration, RAT updates the current estimate $\mathbf{g}_{j-1}$ by projecting the residual $\mathbf{y}_\tau - \mathbf{H}_\tau \mathbf{g}_{j-1}$ onto the solution space of $\mathbf{H}_\tau \mathbf{x} = \mathbf{y}_\tau$, with Tikhonov regularization. This projection implicitly injects curvature information into the advantage estimates. By iterating over mini-batches, RAT progressively aggregates local curvature information, yielding an accurate approximation of the full natural policy gradient without explicitly forming or inverting the Fisher matrix.

Importantly, RAT differs from SPRING (Goldshlager et al., 2024) in two key aspects. First, RAT has no momentum in-

terpretation, whereas SPRING can be viewed as a Kaczmarz-inspired momentum-based method. Second, RAT performs inner iterations over mini-batches within a single on-policy rollout, whereas SPRING applies a single update per batch. This distinction is crucial as it enables iterative refinement of the NPG estimate without cumulating $\mathbf{g}_j$ across rollouts.

As in KFAC (Martens & Grosse, 2015), gradient norm clipping is essential for stability. Instead of clipping in the Fisher norm, we clip ℓ_2 -norm of the estimated gradients $\mathbf{g}_j$ (Zhang et al., 2020): $\alpha_j := \min\left(\eta, \frac{\nu}{\|\mathbf{g}_j\|_2}\right)$, where $\nu > 0$ is a threshold and $\eta > 0$ is the learning rate.

Shared actor-critic architectures. We follow Wu et al. (2017) and estimate joint natural policy gradients by modeling the value function as a Gaussian. To apply RAT to the critic, we explicitly introduce a pseudo advantage (e.g., an all-ones vector). RAT is then applied jointly to the policy advantage and the critic pseudo advantage, yielding a unified and stable optimization objective for shared actor-critic networks. We refer readers to Appendix C.1 for details. This joint application of RAT to both policy and critic updates clearly shows the flexibility of the advantage transformation, and distinguishes our method from prior work on Woodbury-based approaches (Guzmán-Cordero et al., 2025). Specifically, while Guzmán-Cordero et al. (2025) can in principle be applied in this setting, they typically require maintaining separate curvature-adjusted gradients for actor and critic and carefully merging them during parameter updates. This merging is inherently architecture-dependent, as it requires explicit knowledge of which parameters are shared and how gradients from different heads should be combined. In contrast, RAT introduces a pseudo-advantage formulation that unifies the actor and critic objectives into a single surrogate loss. The resulting gradient is computed via standard backpropagation. As a result, curvature-adjusted updates are handled implicitly by `autograd`, without requiring manual gradient partitioning or architecture-specific merging logic. This allows RAT to remain architecture-agnostic in practice, even in shared-network settings. The full RAT is summarized in Algorithm 1 in Appendix.

4.4. Convergence Results of RAT

RAT is closely related to randomized block Kaczmarz method (Needell & Tropp, 2014) and, more generally, to the class of randomized iterative methods (Gower & Richtárik, 2015). Its convergence analysis can be viewed as an extension of these methods to the setting of Tikhonov-regularized natural policy gradients.

We begin with a standard assumption on the matrix $\mathbf{H}$.

Assumption 1 (Full column rank). The full data matrix $\mathbf{H} \in \mathbb{R}^{n \times p}$ satisfies $\text{rank}(\mathbf{H}) = p$ and $p \ll n$.

¹https://docs.pytorch.org/tutorials/intermediate/per_sample_grads.html

This assumption is consistent with the common setting in reinforcement learning in which the state-action space is large or continuous, and function approximation is needed.

Assumption 2 (State-action coverage). Let $\mathbf{h}_i^\top$ denote i -th row of $\mathbf{H}$, and τ denote a random minibatch sampled from $d_\pi(s, a)$. For each index $i \in \{1, \dots, n\}$, $\mathbb{P}(i \in \tau) > 0$.

This assumption ensures that every state-action pair has a non-zero probability of being sampled, which is a standard requirement for defining the natural policy gradient (Bagnell & Schneider, 2003; Kakade, 2001).

Under the above assumptions, we obtain the following:

Lemma 1. Define $\mathbf{P}_\tau := \mathbf{H}_\tau^\top (\lambda \mathbf{I} + \mathbf{H}_\tau \mathbf{H}_\tau^\top)^{-1} \mathbf{H}_\tau$, then

$$\mu := \lambda_{\min}(\mathbb{E}[\mathbf{P}_\tau]) > 0.$$

We now present two theorems characterizing the convergence behavior of RAT. We first analyze an idealized case in which the advantage is *exactly compatible* (Peters & Schaal, 2008). Specifically, for all minibatches τ , $\mathbf{y}_\tau = \mathbf{H}_\tau \mathbf{g}^*$, where $\mathbf{g}^*$ is the solution to Equation (5).

Theorem 1 (Linear convergence of RAT). Assume minibatches τ_j are sampled i.i.d. from $d_\pi(s, a)$. Then

$$\mathbb{E} \|\mathbf{g}_j - \mathbf{g}^*\|_2^2 \leq (1 - \mu)^j \|\mathbf{g}_0 - \mathbf{g}^*\|_2^2. \quad (16)$$

This theorem, together with Lemma 1, guarantees that the RAT update is contractive, which is essential for establishing linear convergence. The convergence rate of RAT is entirely characterized by the spectrum of $\mathbb{E}[\mathbf{P}_\tau]$. In particular, as $\mathbf{P}_\tau = \mathbf{H}_\tau^\top \mathbf{H}_\tau (\lambda \mathbf{I} + \mathbf{H}_\tau^\top \mathbf{H}_\tau)^{-1}$, when $\mathbf{H}_\tau^\top \mathbf{H}_\tau$ (empirical Fisher) has a low rank, a smaller λ generally leads to a larger μ , and thus faster convergence. This behavior is also observed in our sensitivity analysis. In addition, Assumption 1 applies to the *full matrix* $\mathbf{H}$, not to individual minibatches. In practice, minibatch matrices $\mathbf{H}_\tau$ can be low-rank. The Tikhonov damping term $\lambda > 0$ ensures that $(\lambda \mathbf{I} + \mathbf{H}_\tau \mathbf{H}_\tau^\top)$ is always invertible, even when $\mathbf{H}_\tau$ is rank-deficient. Poor state-action coverage affects the convergence rate through $\mu = \lambda_{\min}(\mathbb{E}[\mathbf{P}_\tau])$, but does not invalidate the analysis.

We now consider the more realistic case in which the advantage estimates are noisy: $\mathbf{y}_\tau = \mathbf{H}_\tau \mathbf{g}^* + \boldsymbol{\xi}_\tau$, where $\boldsymbol{\xi}_\tau$ is a zero-mean random variable satisfying $\mathbb{E}[\boldsymbol{\xi}_\tau | \tau] = \mathbf{0}$.

Theorem 2 (Convergence with error floor). Define $\eta^2 := \mathbb{E}[\|\mathbf{H}_\tau^\top (\lambda \mathbf{I} + \mathbf{H}_\tau \mathbf{H}_\tau^\top)^{-1} \boldsymbol{\xi}_\tau\|_2^2]$. Then

$$\mathbb{E} \|\mathbf{g}_j - \mathbf{g}^*\|_2^2 \leq (1 - \mu)^j \|\mathbf{g}_0 - \mathbf{g}^*\|_2^2 + \frac{\eta^2}{\mu}. \quad (17)$$

η^2 effectively quantifies the norm discrepancy between the true gradient and its stochastic estimate. This motivates the use of gradient norm clipping in practice.

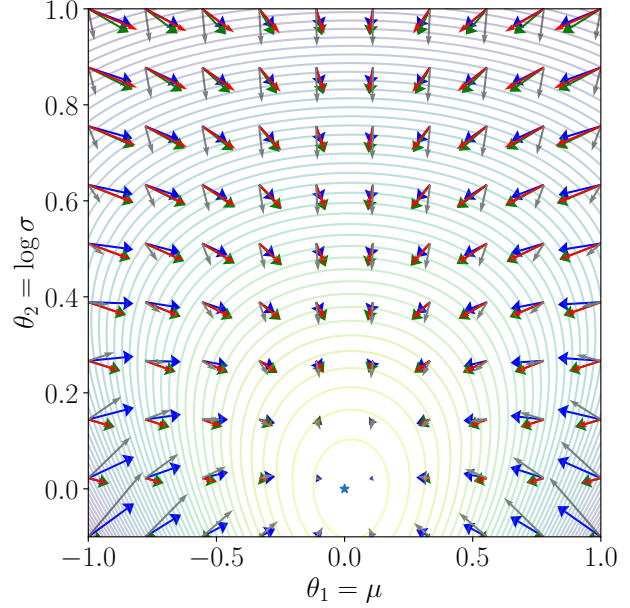


Figure 2. Univariate Gaussian with $\theta_1 = \mu$ and $\theta_2 = \log \sigma$ (closed-formed natural gradients, empirical natural gradients, RAT gradients and vanilla gradients; $\star$ for the optimum). RAT closely approximates empirical natural gradients.

It is worth noting that Algorithm 1 in Appendix implements multiple inner iterations per batch, resulting in a time-varying sequence of linear systems. We show in Appendix C.2 that the practical implementation can be interpreted as a *contractive solver tracking a slowly varying sequence of systems*. Let $\mathbf{g}_t^*$ denote the solution of the regularized least-squares problem defined by the current policy θ_t , and define the tracking error $\mathbf{e}_t := \|\mathbf{g}_t - \mathbf{g}_t^*\|$. Under our settings (small learning rates and gradient clipping), the tracking error remains small, yielding a bounded steady-state error of order $\mathcal{O}(\max_t \|\theta_{t+1} - \theta_t\|)$. This aligns with standard analyses of stochastic approximation in RL, where updates track a moving target induced by policy changes, and provides a heuristic justification for the implementation.

5. Experiments

We evaluate Randomized Advantage Transformation (RAT) through a combination of controlled illustrations, continuous control benchmarks, and high-dimensional visual domains. Our goals are to: (1) verify that RAT accurately approximates empirical natural gradients, (2) assess its empirical performance and efficiency relative to established natural policy gradients methods, and (3) analyze the sensitivity to its key design choices.

Across all experiments, we apply standard stabilization techniques, including observation normalization (Mnih et al., 2016), advantage normalization (Schulman et al., 2017b),

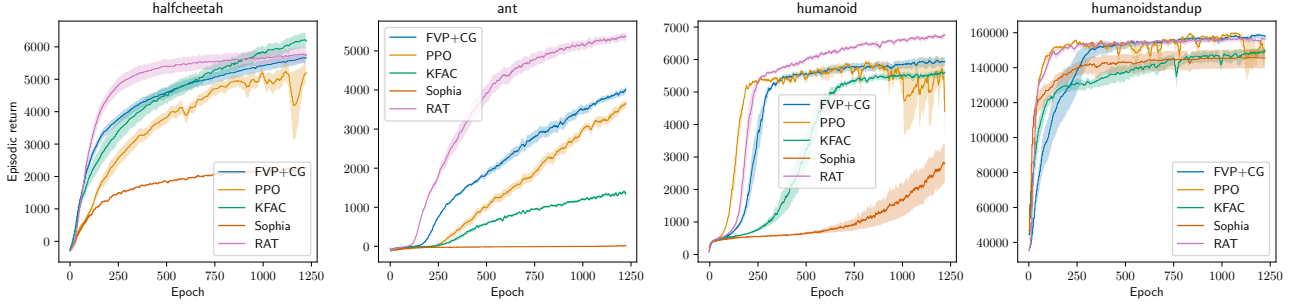


Figure 3. Optimizing MLP policies on continuous control tasks with separate actor-critic networks. RAT outperforms KFAC, FVP+CG and Sophia in most tasks. The shaded region denotes the standard error over 5 random seeds.

Table 1. Final performance (mean $\pm$ stderr over 5 seeds) of different methods on continuous controls with shared actor-critic networks.

Ep. Returns	Swimmer $\uparrow$	Hopper $\uparrow$	HalfCheetah $\uparrow$	Walker2d $\uparrow$	Ant $\uparrow$	Humanoid $\uparrow$	HumanoidStandup $\uparrow$
$S \times \mathcal{A}$	8×2	11×3	17×6	17×6	105×8	376×17	376×17
RAT (Ours)	271.6$\pm$36.3	2334.6 $\pm$ 524.9	4629.2$\pm$287.4	3156.0$\pm$293.6	2926.6$\pm$353.1	5382.7$\pm$117.3	146529.7$\pm$2317.6
ACKTR	59.1 $\pm$ 13.0	2138.9 $\pm$ 171.6	3630.9 $\pm$ 282.6	2576.6 $\pm$ 154.6	23.4 $\pm$ 3.2	2571.7 $\pm$ 838.7	127928.5 $\pm$ 5433.7
PPO	191.3 $\pm$ 32.7	2346.8$\pm$202.7	4146.0 $\pm$ 107.5	2225.3 $\pm$ 303.4	1373.9 $\pm$ 26.0	5357.9 $\pm$ 150.9	130014.2 $\pm$ 6463.7
Sophia	57.9 $\pm$ 5.9	1104.0 $\pm$ 90.6	899.5 $\pm$ 113.2	1256.0 $\pm$ 129.7	-7.0 $\pm$ 1.4	669.4 $\pm$ 56.2	111212.6 $\pm$ 13449.9

and PopArt for value normalization (Hessel et al., 2019). Unless stated otherwise, all methods use the same network architectures and training pipelines. We report results averaged over five random seeds for a fixed training budget of 1250 epochs, which corresponds to approximately 10 million environment steps (a standard budget in continuous control benchmarks). Additional implementation details are provided in Appendix C. The code for reproducing our results is available at [Code URL](https://github.com/agent-lab/ICML2026-RAT)².

5.1. Illustration of Natural Gradients Estimation

We begin with a low-dimensional example that admits an analytic form of the natural gradient, enabling direct visualization of the update directions. Specifically, we consider maximum-likelihood estimation for a univariate Gaussian parameterized by its mean and log-standard deviation: $\mathcal{N}(x|\mu, \sigma; \theta_1, \theta_2)$, where $\mu = \theta_1$ and log standard deviation $\log \sigma = \theta_2$. In this setting, the inverse Fisher matrix is available in closed form, allowing us to compute exact natural gradients (see Appendix A for details). Figure 2 compares the vanilla gradients, the natural gradient, the empirical natural gradients and the gradient estimated by RAT. We also plot the contour lines for $\log \mathcal{N}(x|\theta)$ (loss landscape) to better illustrate how the natural gradient differs from the vanilla gradient. While vanilla gradients follow the steepest ascent direction of $\log \mathcal{N}(x|\theta)$, perpendicular to the contour lines, the natural gradient accounts for the geometry induced by the parameterization and points more directly towards the

optimum. The updates produced by RAT closely match the empirical natural gradients, demonstrating the effectiveness and accuracy of RAT with finite samples.

5.2. Continuous Control with MLP Policies

We next evaluate RAT on standard continuous control benchmarks from OpenAI Gym (Brockman et al., 2016) implemented in MuJoCo (Todorov et al., 2012). We consider Walker2d-v4 ($\mathcal{A} \in \mathbb{R}^6$), HalfCheetah-v4 ($\mathcal{A} \in \mathbb{R}^6$), Ant-v4 ($\mathcal{A} \in \mathbb{R}^8$), and Humanoid-v4 ($\mathcal{A} \in \mathbb{R}^{17}$), which span action dimensions from 6 to 17. Policies and value functions are parameterized by two-layer MLPs with 256 hidden units and Tanh activations; the policy outputs the mean of a Gaussian distribution, with a state-independent log standard deviation. We compare RAT against several strong baselines for estimating natural policy gradients, including Fisher-vector products with conjugate gradient (FVP+CG) from (Schulman et al., 2015), Kronecker-Factored Approximate Curvature (KFAC) (Martens & Grosse, 2015), and a diagonal Fisher approximation (i.e., $\text{diag}(\lambda \mathbf{I} + \mathbf{H}^\top \mathbf{H})$) according to Sophia (Liu et al., 2024). All methods are implemented within the same codebase, and baseline hyperparameters are tuned for best performance.

Separate actor-critic networks. Figure 3 reports learning curves when actor and critic are optimized separately. RAT consistently matches or outperforms all baselines across all tasks, exhibiting both faster learning and higher final returns. In particular, RAT remains stable on challenging tasks such as Ant-v4 and Humanoid-v4, whereas KFAC frequently

²<https://github.com/agent-lab/ICML2026-RAT>

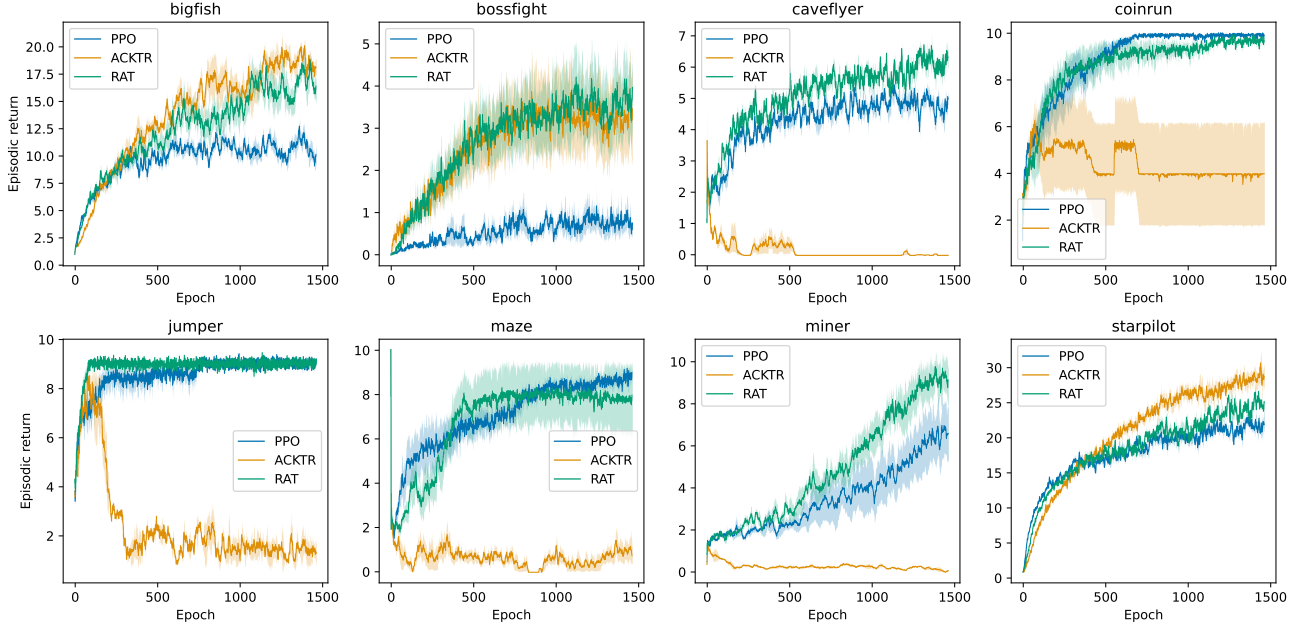


Figure 4. Optimizing ResNet policies for discrete controls in ProcGen environments: RAT performs consistently well across all 8 tasks, delivering comparable or higher episodic returns than all baselines. The shaded region denotes the standard error over 5 random seeds.

Table 2. Wallclock time per update (in ms) on continuous control tasks with separate actor-critic networks.

	Time (ms)	HalfCheetah ↓	Ant ↓	Humanoid ↓
Separate	RAT (Ours)	9.83±1.49	10.04±1.35	18.17±3.04
	FVP+CG	19.86±1.15	19.95±1.20	19.81±1.18
	KFAC	5.60±1.28	5.61±1.23	6.57±1.47
	Sophia	3.92±0.71	3.98±0.73	5.71±0.68
	PPO	3.12±1.38	3.18±1.36	3.22±1.40
Shared	RAT (Ours)	11.53±1.69	11.66±1.55	19.85±3.11
	ACKTR	6.92±1.63	6.85±1.55	7.87±1.70
	Sophia	5.97±0.94	6.03±1.02	7.58±0.97
	PPO	3.70±1.56	3.70±1.46	3.72±1.49

learns slowly. We also evaluated an enhanced variant of KFAC, i.e., eKFAC (George et al., 2018), and found that the eKFAC method did not yield noticeable improvements over KFAC in our experiments, see Figure 6 in Appendix E. The Sophia method performs poorly on all tasks, highlighting the importance of capturing parameter correlations. In addition, RAT performs significantly better than PPO on challenging Ant and Humanoid tasks.

Shared actor-critic networks. We further evaluate RAT in the shared-network setting, where curvature estimation is more challenging. Since FVP+CG is not directly applicable, we compare against Proximal Policy Optimization (PPO) (Schulman et al., 2017b), a simplified approximation to natural policy gradients (Hilton et al., 2022), ACKTR (Wu

et al., 2017), an extended KFAC method for shared networks, and Sophia (Liu et al., 2024). Table 1 summarizes final performance. RAT achieves the best overall returns on most tasks, with substantial gains on Ant and Humanoid, highlighting its robustness in challenging shared architectures.

Table 2 reports wallclock time per update (in ms; averaged over 124 updates on Xeon(R) w5-2445 GeForce RTX 4090). While the PPO is the fastest, RAT is significantly more efficient than FVP+CG and offers a favorable trade-off between computational cost and performance. Despite higher per-update cost, RAT is most beneficial in regimes where curvature matters (e.g., high-dimensional settings), where PPO often plateaus or requires careful tuning. RAT is not designed to match PPO’s per-step efficiency, but to provide a simple, architecture-agnostic, and principled approximation to natural policy gradients. Compared to existing natural-gradient methods, it offers a stronger performance–compute trade-off while avoiding architecture-specific approximations and complex inner solvers.

5.3. Visual Control with ResNet Policies

To assess scalability to high-dimensional visual inputs, we evaluate RAT on the challenging Procgen Benchmark (Cobbe et al., 2020), which features procedurally generated environments with 64x64 RGB observations. We consider 8 representative environments, including *BigFish*, *BossFight*, *CaveFlyer*, *Climber*, *Dodgeball*, *FruitBot*, *Heist*, and *StarPilot*. Policies are parameterized using a ResNet-

based architecture, adapted from Espeholt et al. (2018), and training follows the standard Progen protocol for evaluating sample efficiency, i.e., training and testing on the same distribution of levels in each environment.

Figure 4 presents learning curves comparing RAT with PPO and ACKTR. RAT performs consistently well across all tasks, delivering comparable or higher returns than all baselines while avoiding training instabilities frequently observed in KFAC-based methods. These results demonstrate that RAT scales effectively to visual and complex dynamics.

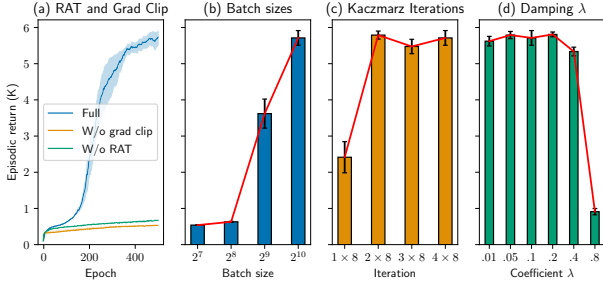


Figure 5. Ablation and sensitivity analysis of RAT on Humanoid.

5.4. Ablations and Sensitivity Analysis

Finally, we conduct an ablation study and sensitivity analysis to pinpoint the influence of key components and hyperparameters of RAT on performance, focusing on the challenging Humanoid task. Figure 5(a) presents ablations that remove either the advantage transformation or gradient norm clipping. Both components are essential: removing either leads to substantial performance degradation. We further study sensitivity of RAT to batch size, number of Kaczmarz iterations and damping coefficient λ . The results are presented in Figure 5(b), (c) and (d). RAT benefits from sufficiently large batch sizes (2^{10}) (as the batch size effectively determines the rank of empirical Fisher matrix), while remaining relatively robust to the number of Kaczmarz iterations within a reasonable range (too few iterations result in suboptimal performance). The performance of RAT is also robust across a broad range of damping values (from 0.01 to 0.4), with degradation only occurring at very large values (e.g., $\lambda = 0.8$). More analysis on Ant can be found in Figure 8 in Appendix. Overall, these results indicate that RAT is robust and does not require fine-grained tuning.

6. Limitations and Conclusion

Limitations. Despite strong theoretical and empirical results, RAT has several limitations. First, RAT requires forming minibatch-level matrices $\mathbf{H}_\tau \mathbf{H}_\tau^\top$, with cost $\mathcal{O}(pB^2)$. While substantially cheaper than full Fisher inversion and architecture-agnostic, this can become a bottleneck for large policies or batch sizes; low-rank or sketch-based approxi-

mations may improve scalability. Second, the convergence rate of RAT depends on the minimum singular value of $\mathbf{P}_\tau$. Poorly conditioned minibatches may slow convergence, although gradient norm clipping alleviates this issue in practice. Third, our analysis focus on the on-policy setting; extending RAT to full off-policy settings where the advantage function is estimated from replay buffer samples remains an open direction.

Conclusion. We proposed Randomized Advantage Transformation (RAT), an efficient and architecture-agnostic method for estimating natural policy gradients. By using a Woodbury-based reformulation, RAT transforms curvature information into the advantage function and estimates regularized natural policy gradients using standard backpropagation, without explicit Fisher construction or conjugate-gradient solvers. We provided convergence guarantees and demonstrated strong empirical performance on continuous control and high-dimensional visual benchmarks. RAT bridges the gap between principled natural gradient methods and practical deep reinforcement learning, offering a simple and scalable alternative for second-order policy optimization.

Acknowledgements

This work was supported in part by the Engineering and Physical Sciences Research Council (EPSRC) through the AI Hub in Generative Models [grant number EP/Y028805/1]. The author would like to thank the anonymous reviewers for their valuable feedback and suggestions.

Impact Statement

This work advances scalable optimization methods for reinforcement learning by enabling efficient computation of natural policy gradients without explicit curvature estimation. By simplifying implementation and reducing computational overhead, the proposed method may facilitate broader adoption of principled second-order optimization techniques in practice. As with reinforcement learning methods in general, potential downstream applications span a wide range of domains and should be deployed responsibly, particularly in efficiency-critical settings. This paper focuses on algorithmic contributions and does not involve human subjects or sensitive data.

References

Agarwal, A., Kakade, S. M., Lee, J. D., and Mahajan, G. On the theory of policy gradient methods: Optimality, approximation, and distribution shift. *J. Mach. Learn. Res.*, 22:98:1–98:76, 2021. URL <https://jmlr.org/papers/v22/19-736.html>.

- Amari, S. Natural gradient works efficiently in learning. *Neural Comput.*, 10(2):251–276, 1998. doi: 10.1162/089976698300017746. URL <https://doi.org/10.1162/089976698300017746>.
- Amari, S., Park, H., and Fukumizu, K. Adaptive method of realizing natural gradient learning for multilayer perceptrons. *Neural Comput.*, 12(6):1399–1409, 2000. doi: 10.1162/089976600300015420. URL <https://doi.org/10.1162/089976600300015420>.
- Ba, J., Grosse, R. B., and Martens, J. Distributed second-order optimization using kronecker-factored approximations. In *5th International Conference on Learning Representations, ICLR 2017, Toulon, France, April 24-26, 2017, Conference Track Proceedings*. OpenReview.net, 2017. URL <https://openreview.net/forum?id=SkkTMPjex>.
- Bae, J., Vicol, P., HaoChen, J. Z., and Grosse, R. B. Amortized proximal optimization. In Koyejo, S., Mohamed, S., Agarwal, A., Belgrave, D., Cho, K., and Oh, A. (eds.), *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022*, 2022. URL http://papers.nips.cc/paper_files/paper/2022/hash/3af25aa3de8b7b02ddbd1b6be5031be8-Abstract-Conference.html.
- Bagnell, J. A. and Schneider, J. Covariant policy search. In *Proceedings of the 18th International Joint Conference on Artificial Intelligence, IJCAI’03*, pp. 1019–1024, San Francisco, CA, USA, 2003. Morgan Kaufmann Publishers Inc.
- Benzing, F. Gradient descent on neurons and its link to approximate second-order optimization. In Chaudhuri, K., Jegelka, S., Song, L., Szepesvári, C., Niu, G., and Sabato, S. (eds.), *International Conference on Machine Learning, ICML 2022, 17-23 July 2022, Baltimore, Maryland, USA*, volume 162 of *Proceedings of Machine Learning Research*, pp. 1817–1853. PMLR, 2022. URL <https://proceedings.mlr.press/v162/benzing22a.html>.
- Brockman, G., Cheung, V., Pettersson, L., Schneider, J., Schulman, J., Tang, J., and Zaremba, W. Openai gym. *CoRR*, abs/1606.01540, 2016. URL <http://arxiv.org/abs/1606.01540>.
- Cayci, S. and Eryilmaz, A. Recurrent natural policy gradient for pomdps. *Trans. Mach. Learn. Res.*, 2025, 2025. URL <https://openreview.net/forum?id=6G01e0vgIf>.
- Chen, A. and Heyl, M. Empowering deep neural quantum states through efficient optimization. *Nature Physics*, 20(9):1476–1481, 2024.
- Cobbe, K., Hesse, C., Hilton, J., and Schulman, J. Leveraging procedural generation to benchmark reinforcement learning. In *Proceedings of the 37th International Conference on Machine Learning, ICML 2020, 13-18 July 2020, Virtual Event*, volume 119 of *Proceedings of Machine Learning Research*, pp. 2048–2056. PMLR, 2020. URL <http://proceedings.mlr.press/v119/cobbe20a.html>.
- Dangel, F., Tatzel, L., and Hennig, P. Vivit: Curvature access through the generalized gauss-newton’s low-rank structure. *Trans. Mach. Learn. Res.*, 2023, 2023. URL <https://openreview.net/forum?id=DzJ7JfPXkE>.
- Espeholt, L., Soyer, H., Munos, R., Simonyan, K., Mnih, V., Ward, T., Doron, Y., Firoiu, V., Harley, T., Dunning, I., Legg, S., and Kavukcuoglu, K. IMPALA: scalable distributed deep-rl with importance weighted actor-learner architectures. In Dy, J. G. and Krause, A. (eds.), *Proceedings of the 35th International Conference on Machine Learning, ICML 2018, Stockholm, Sweden, July 10-15, 2018*, volume 80 of *Proceedings of Machine Learning Research*, pp. 1406–1415. PMLR, 2018. URL <http://proceedings.mlr.press/v80/espeholt18a.html>.
- George, T., Laurent, C., Bouthillier, X., Ballas, N., and Vincent, P. Fast approximate natural gradient descent in a kronecker factored eigenbasis. In Bengio, S., Wallach, H. M., Larochelle, H., Grauman, K., Cesa-Bianchi, N., and Garnett, R. (eds.), *Advances in Neural Information Processing Systems 31: Annual Conference on Neural Information Processing Systems 2018, NeurIPS 2018, December 3-8, 2018, Montréal, Canada*, pp. 9573–9583, 2018. URL <https://proceedings.neurips.cc/paper/2018/hash/48000647b315f6f00f913caa757a70b3-Abstract.html>.
- Gittens, A. and Mahoney, M. Revisiting the nystrom method for improved large-scale machine learning. In Dasgupta, S. and McAllester, D. (eds.), *Proceedings of the 30th International Conference on Machine Learning*, volume 28 of *Proceedings of Machine Learning Research*, pp. 567–575, Atlanta, Georgia, USA, 17–19 Jun 2013. PMLR. URL <https://proceedings.mlr.press/v28/gittens13.html>.
- Goldshlager, G., Abrahamsen, N., and Lin, L. A kaczmarz-inspired approach to accelerate the optimization of neural network wavefunctions. *Journal of Computational Physics*, 516:113351, 2024.

- Goldshlager, G., Hu, J., and Lin, L. A sketch-and-project analysis of subsampled natural gradient algorithms, 2026. URL <https://arxiv.org/abs/2508.21022>.
- Gower, R. M. and Richtárik, P. Randomized iterative methods for linear systems. *SIAM J. Matrix Anal. Appl.*, 36 (4):1660–1690, 2015. doi: 10.1137/15M1025487. URL <https://doi.org/10.1137/15M1025487>.
- Guzmán-Cordero, A., Dangel, F., Goldshlager, G., and Zeinhofer, M. Improving energy natural gradient descent through woodbury, momentum, and randomization. *CoRR*, abs/2505.12149, 2025. doi: 10.48550/ARXIV.2505.12149. URL <https://doi.org/10.48550/arXiv.2505.12149>.
- Haarnoja, T., Zhou, A., Abbeel, P., and Levine, S. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In Dy, J. G. and Krause, A. (eds.), *Proceedings of the 35th International Conference on Machine Learning, ICML 2018, Stockholm, Sweden, July 10-15, 2018*, volume 80 of *Proceedings of Machine Learning Research*, pp. 1856–1865. PMLR, 2018. URL <http://proceedings.mlr.press/v80/haarnoja18b.html>.
- Hessel, M., Soyer, H., Espeholt, L., Czarnecki, W., Schmitt, S., and van Hasselt, H. Multi-task deep reinforcement learning with popart. In *Proceedings of the Thirty-Third AAAI Conference on Artificial Intelligence and Thirty-First Innovative Applications of Artificial Intelligence Conference and Ninth AAAI Symposium on Educational Advances in Artificial Intelligence, AAAI’19/IAAI’19/EAAI’19*. AAAI Press, 2019. ISBN 978-1-57735-809-1. doi: 10.1609/aaai.v33i01.33013796. URL <https://doi.org/10.1609/aaai.v33i01.33013796>.
- Hilton, J., Cobbe, K., and Schulman, J. Batch size-invariance for policy optimization. In Oh, A. H., Agarwal, A., Belgrave, D., and Cho, K. (eds.), *Advances in Neural Information Processing Systems*, 2022. URL <https://openreview.net/forum?id=1XuZaxEaI7>.
- Huo, Y., Dash, S. P., Stoican, R., Kaski, S., and Sun, M. Rank-1 approximation of inverse fisher for natural policy gradients in deep reinforcement learning. *Transactions on Machine Learning Research*, 2026. ISSN 2835-8856. URL <https://openreview.net/forum?id=ko8Kn7TS6m>.
- Jacot, A., Hongler, C., and Gabriel, F. Neural tangent kernel: Convergence and generalization in neural networks. In Bengio, S., Wallach, H. M., Larochelle, H., Grauman, K., Cesa-Bianchi, N., and Garnett, R. (eds.), *Advances in Neural Information Processing Systems 31: Annual Conference on Neural Information Processing Systems 2018, NeurIPS 2018, December 3-8, 2018, Montréal, Canada*, pp. 8580–8589, 2018. URL <https://proceedings.neurips.cc/paper/2018/hash/5a4belfa34e62bb8a6ec6b91d2462f5a-Abstract.html>.
- Kakade, S. M. A natural policy gradient. In Dietterich, T. G., Becker, S., and Ghahramani, Z. (eds.), *Advances in Neural Information Processing Systems 14 [Neural Information Processing Systems: Natural and Synthetic, NIPS 2001, December 3-8, 2001, Vancouver, British Columbia, Canada]*, pp. 1531–1538. MIT Press, 2001. URL <https://proceedings.neurips.cc/paper/2001/hash/4b86abe48d358ecf194c56c69108433e-Abstract.html>.
- Kingma, D. P. and Ba, J. Adam: A method for stochastic optimization. In Bengio, Y. and LeCun, Y. (eds.), *3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, Conference Track Proceedings*, 2015. URL <http://arxiv.org/abs/1412.6980>.
- Kuba, J. G., Chen, R., Wen, M., Wen, Y., Sun, F., Wang, J., and Yang, Y. Trust region policy optimisation in multi-agent reinforcement learning. In *The Tenth International Conference on Learning Representations, ICLR 2022, Virtual Event, April 25-29, 2022*. OpenReview.net, 2022. URL <https://openreview.net/forum?id=EcGGfKNTxdJ>.
- Kunstner, F., Hennig, P., and Balles, L. Limitations of the empirical fisher approximation for natural gradient descent. In Wallach, H. M., Larochelle, H., Beygelzimer, A., d’Alché-Buc, F., Fox, E. B., and Garnett, R. (eds.), *Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019, December 8-14, 2019, Vancouver, BC, Canada*, pp. 4158–4169, 2019. URL <https://proceedings.neurips.cc/paper/2019/hash/46a558d97954d0692411c861cf78ef79-Abstract.html>.
- Liu, H., Li, Z., Hall, D. L. W., Liang, P., and Ma, T. Sophia: A scalable stochastic second-order optimizer for language model pre-training. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024. URL <https://openreview.net/forum?id=3xHDeA8Noi>.
- Martens, J. Deep learning via hessian-free optimization. In Fürnkranz, J. and Joachims, T. (eds.), *Proceedings of the 27th International Conference on Machine Learning (ICML-10), June 21-24, 2010, Haifa, Israel*, pp. 735–742. Omnipress, 2010. URL <https://icml.cc/Conferences/2010/papers/458.pdf>.

- Martens, J. and Grosse, R. B. Optimizing neural networks with kronecker-factored approximate curvature. In Bach, F. R. and Blei, D. M. (eds.), *Proceedings of the 32nd International Conference on Machine Learning, ICML 2015, Lille, France, 6-11 July 2015*, volume 37 of *JMLR Workshop and Conference Proceedings*, pp. 2408–2417. JMLR.org, 2015. URL <http://proceedings.mlr.press/v37/martens15.html>.
- Mnih, V., Badia, A. P., Mirza, M., Graves, A., Lillicrap, T. P., Harley, T., Silver, D., and Kavukcuoglu, K. Asynchronous methods for deep reinforcement learning. In Balcan, M. and Weinberger, K. Q. (eds.), *Proceedings of the 33rd International Conference on Machine Learning, ICML 2016, New York City, NY, USA, June 19-24, 2016*, volume 48 of *JMLR Workshop and Conference Proceedings*, pp. 1928–1937. JMLR.org, 2016. URL <http://proceedings.mlr.press/v48/mnih16.html>.
- Needell, D. and Tropp, J. A. Paved with good intentions: analysis of a randomized block kaczmarz method. *Linear Algebra and its Applications*, 441:199–221, 2014.
- Novak, R., Sohl-Dickstein, J., and Schoenholz, S. S. Fast finite width neural tangent kernel. In Chaudhuri, K., Jegelka, S., Song, L., Szepesvári, C., Niu, G., and Sabato, S. (eds.), *International Conference on Machine Learning, ICML 2022, 17-23 July 2022, Baltimore, Maryland, USA*, volume 162 of *Proceedings of Machine Learning Research*, pp. 17018–17044. PMLR, 2022. URL <https://proceedings.mlr.press/v162/novak22a.html>.
- Pascanu, R. and Bengio, Y. Revisiting natural gradient for deep networks. In Bengio, Y. and LeCun, Y. (eds.), *2nd International Conference on Learning Representations, ICLR 2014, Banff, AB, Canada, April 14-16, 2014, Conference Track Proceedings*, 2014. URL <http://arxiv.org/abs/1301.3584>.
- Peters, J. and Schaal, S. Natural actor-critic. *Neurocomputing*, 71(7-9):1180–1190, 2008. doi: 10.1016/J.NEUCOM.2007.11.026. URL <https://doi.org/10.1016/j.neucom.2007.11.026>.
- Schulman, J., Levine, S., Abbeel, P., Jordan, M. I., and Moritz, P. Trust region policy optimization. In Bach, F. R. and Blei, D. M. (eds.), *Proceedings of the 32nd International Conference on Machine Learning, ICML 2015, Lille, France, 6-11 July 2015*, volume 37 of *JMLR Workshop and Conference Proceedings*, pp. 1889–1897. JMLR.org, 2015. URL <http://proceedings.mlr.press/v37/schulman15.html>.
- Schulman, J., Moritz, P., Levine, S., Jordan, M. I., and Abbeel, P. High-dimensional continuous control using generalized advantage estimation. In Bengio, Y. and LeCun, Y. (eds.), *4th International Conference on Learning Representations, ICLR 2016, San Juan, Puerto Rico, May 2-4, 2016, Conference Track Proceedings*, 2016. URL <http://arxiv.org/abs/1506.02438>.
- Schulman, J., Abbeel, P., and Chen, X. Equivalence between policy gradients and soft q-learning. *CoRR*, abs/1704.06440, 2017a. URL <http://arxiv.org/abs/1704.06440>.
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. Proximal policy optimization algorithms. *CoRR*, abs/1707.06347, 2017b. URL <http://arxiv.org/abs/1707.06347>.
- Sun, M., Devlin, S., Beck, J., Hofmann, K., and Whiteson, S. Trust region bounds for decentralized PPO under non-stationarity. In Agmon, N., An, B., Ricci, A., and Yeoh, W. (eds.), *Proceedings of the 2023 International Conference on Autonomous Agents and Multiagent Systems, AAMAS 2023, London, United Kingdom, 29 May 2023 - 2 June 2023*, pp. 5–13. ACM, 2023. doi: 10.5555/3545946.3598613. URL <https://dl.acm.org/doi/10.5555/3545946.3598613>.
- Sutton, R. S. and Barto, A. G. *Reinforcement learning - an introduction, 2nd Edition*. MIT Press, 2018. URL <http://www.incompleteideas.net/book/thead-book-2nd.html>.
- Sutton, R. S., McAllester, D. A., Singh, S., and Mansour, Y. Policy gradient methods for reinforcement learning with function approximation. In Solla, S. A., Leen, T. K., and Müller, K. (eds.), *Advances in Neural Information Processing Systems 12, [NIPS Conference, Denver, Colorado, USA, November 29 - December 4, 1999]*, pp. 1057–1063. The MIT Press, 1999. URL <http://papers.nips.cc/paper/1713-policy-gradient-methods-for-reinforcement-learning-with-function-approximation>.
- Todorov, E., Erez, T., and Tassa, Y. Mujoco: A physics engine for model-based control. In *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems, IROS 2012, Vilamoura, Algarve, Portugal, October 7-12, 2012*, pp. 5026–5033. IEEE, 2012. doi: 10.1109/IROS.2012.6386109. URL <https://doi.org/10.1109/IROS.2012.6386109>.
- Wu, X., Yu, W., Zhang, C., and Woodland, P. C. An improved empirical fisher approximation for natural gradient descent. In Globersons, A., Mackey, L., Belgrave, D., Fan, A., Paquet, U., Tomczak, J. M., and Zhang, C. (eds.), *Advances in Neural Information Processing*

Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024, 2024. URL http://papers.nips.cc/paper_files/paper/2024/hash/f23098fa0cfcdef0e743b134d380eeb9-Abstract-Conference.html.

Wu, Y., Mansimov, E., Grosse, R. B., Liao, S., and Ba, J. Scalable trust-region method for deep reinforcement learning using kronecker-factored approximation. In Guyon, I., von Luxburg, U., Bengio, S., Wallach, H. M., Fergus, R., Vishwanathan, S. V. N., and Garnett, R. (eds.), *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4-9, 2017, Long Beach, CA, USA*, pp. 5279–5288, 2017. URL <https://proceedings.neurips.cc/paper/2017/hash/361440528766bbaaaa1901845cf4152b-Abstract.html>.

Yang, M., Xu, D., Wen, Z., Chen, M., and Xu, P. Sketch-based empirical natural gradient methods for deep learning. *J. Sci. Comput.*, 92(3):94, 2022. doi: 10.1007/S10915-022-01911-X. URL <https://doi.org/10.1007/s10915-022-01911-x>.

Zhang, J., He, T., Sra, S., and Jadbabaie, A. Why gradient clipping accelerates training: A theoretical justification for adaptivity. In *8th International Conference on Learning Representations, ICLR 2020, Addis Ababa, Ethiopia, April 26-30, 2020*. OpenReview.net, 2020. URL <https://openreview.net/forum?id=BJgnXpVYwS>.

A. Gradient Calculation in the Likelihood Example

For the likelihood example, the PG and NPG are given in the closed forms as follows:

$$\underbrace{\nabla_{\theta} \mathbb{E}_x [\log \mathcal{N}(x|\theta)] = \mathbb{E}_x \left[\begin{array}{c} \frac{(x-\theta_1)}{\exp(2\theta_2)} \\ -1 + \frac{(x-\theta_1)^2}{\exp(2\theta_2)} \end{array} \right]}_{\text{Vanilla gradient}}, \quad \underbrace{F^{-1}(\theta) \nabla_{\theta} \mathbb{E}_x [\log \mathcal{N}(x|\theta)] = \mathbb{E}_x \left[\begin{array}{c} (x-\theta_1) \\ -\frac{1}{2} + \frac{(x-\theta_1)^2}{2\exp(2\theta_2)} \end{array} \right]}_{\text{Natural gradient}}$$

Specifically, with parameterization $\theta_1 = \mu$ and $\theta_2 = \log \sigma$, we have

$$\mathcal{N}(x|\mu, \sigma; \theta) = \frac{1}{\sqrt{2\pi}\sigma} \exp \left[-\frac{(x-\mu)^2}{2\sigma^2} \right] = \frac{1}{\sqrt{2\pi} \exp(\theta_2)} \exp \left[-\frac{(x-\theta_1)^2}{2\exp(2\theta_2)} \right], \quad (18)$$

$$\log \mathcal{N}(x|\mu, \sigma; \theta) = -\log \sqrt{2\pi} - \theta_2 - \frac{(x-\theta_1)^2}{2\exp(2\theta_2)}. \quad (19)$$

The gradient of log probability is given as

$$\nabla_{\theta} \log \mathcal{N}(x|\theta) = \begin{bmatrix} \exp(-2\theta_2)(x-\theta_1) \\ -1 + \exp(-2\theta_2)(x-\theta_1)^2 \end{bmatrix} = \begin{bmatrix} \frac{(x-\theta_1)}{\sigma^2} \\ -1 + \frac{(x-\theta_1)^2}{\sigma^2} \end{bmatrix}. \quad (20)$$

Accordingly, the Fisher matrix at point x is given

$$F(x, \theta) = \begin{bmatrix} \frac{(x-\theta_1)^2}{\sigma^4} - \frac{(x-\theta_1)}{\sigma^2} & \frac{(x-\theta_1)^3}{\sigma^4} - \frac{(x-\theta_1)}{\sigma^2} \\ \frac{(x-\theta_1)^3}{\sigma^4} - \frac{(x-\theta_1)}{\sigma^2} & 1 - \frac{2(x-\theta_1)^2}{\sigma^2} + \frac{(x-\theta_1)^4}{\sigma^4} \end{bmatrix}. \quad (21)$$

Thus, the full Fisher matrix is

$$F(\theta) = \mathbb{E}_{x \sim \mathcal{N}(x|\mu, \sigma, \theta)} \begin{bmatrix} \frac{(x-\theta_1)^2}{\sigma^4} & \frac{(x-\theta_1)^3}{\sigma^4} - \frac{(x-\theta_1)}{\sigma^2} \\ \frac{(x-\theta_1)^3}{\sigma^4} - \frac{(x-\theta_1)}{\sigma^2} & 1 - \frac{2(x-\theta_1)^2}{\sigma^2} + \frac{(x-\theta_1)^4}{\sigma^4} \end{bmatrix} \quad (22)$$

$$= \begin{bmatrix} \mathbb{E}_{x \sim \mathcal{N}(x|\mu, \sigma; \theta)} \left[\frac{(x-\theta_1)^2}{\sigma^4} \right] & \mathbb{E}_{x \sim \mathcal{N}(x|\mu, \sigma; \theta)} \left[\frac{(x-\theta_1)^3}{\sigma^4} - \frac{(x-\theta_1)}{\sigma^2} \right] \\ \mathbb{E}_{x \sim \mathcal{N}(x|\mu, \sigma; \theta)} \left[\frac{(x-\theta_1)^3}{\sigma^4} - \frac{(x-\theta_1)}{\sigma^2} \right] & \mathbb{E}_{x \sim \mathcal{N}(x|\mu, \sigma; \theta)} \left[1 - \frac{2(x-\theta_1)^2}{\sigma^2} + \frac{(x-\theta_1)^4}{\sigma^4} \right] \end{bmatrix} \quad (23)$$

$$= \begin{bmatrix} \frac{1}{\sigma^2} & 0 \\ 0 & 2 \end{bmatrix} = \begin{bmatrix} \exp(-2\theta_2) & 0 \\ 0 & 2 \end{bmatrix}. \quad (24)$$

The transition from Equation (23) to Equation (24) is based on the fact that if x has a normal distribution $\mathcal{N}(x|\mu, \sigma)$, the non-central moments exist for any non-negative integer p and are given as follows:

$$\mathbb{E}_x [(x-\mu)^p] = \begin{cases} 0 & \text{if } p \text{ is odd,} \\ \sigma^p (p-1)!! & \text{if } p \text{ is even.} \end{cases} \quad (25)$$

For a diagonal Fisher matrix, its inverse exists since $\exp(2\theta_2) > 0$ and is given below

$$F^{-1}(\theta) = \begin{bmatrix} \sigma^2 & 0 \\ 0 & \frac{1}{2} \end{bmatrix} = \begin{bmatrix} \exp(2\theta_2) & 0 \\ 0 & \frac{1}{2} \end{bmatrix}. \quad (26)$$

Thus, the vanilla gradient direction for the parameter update is

$$\nabla_{\theta} \mathbb{E}_x [\log p(x|\theta)] = \mathbb{E}_x \left[\begin{array}{c} \frac{(x-\theta_1)}{\exp(2\theta_2)} \\ -1 + \frac{(x-\theta_1)^2}{\exp(2\theta_2)} \end{array} \right], \quad (27)$$

and the natural gradient direction is

$$F^{-1}(\theta) \mathbb{E}_x [\nabla_{\theta} p(x|\theta)] = \mathbb{E}_x [F^{-1}(\theta) \nabla_{\theta} p(x|\theta)] = \mathbb{E}_x \left[\begin{array}{c} (x-\theta_1) \\ -\frac{1}{2} + \frac{(x-\theta_1)^2}{2\exp(2\theta_2)} \end{array} \right] \quad (28)$$

For damped Fisher $\lambda I + F(\theta)$, its inverse is given by

$$(\lambda I + F(\theta))^{-1} = \begin{bmatrix} \frac{\sigma^2}{1+\lambda\sigma^2} & 0 \\ 0 & \frac{1}{2+\lambda} \end{bmatrix} = \begin{bmatrix} \frac{\exp(2\theta_2)}{1+\lambda \exp(2\theta_2)} & 0 \\ 0 & \frac{1}{2+\lambda} \end{bmatrix}. \quad (29)$$

The regularized natural gradient direction is

$$(\lambda I + F(\theta))^{-1} \mathbb{E}_x [\nabla_\theta p(x|\theta)] = \mathbb{E}_x \left[\begin{bmatrix} \frac{x-\theta_1}{1+\lambda \exp(2\theta_2)} \\ -\frac{1}{2+\lambda} + \frac{(x-\theta_1)^2}{(2+\lambda) \exp(2\theta_2)} \end{bmatrix} \right] \quad (30)$$

To compute the gradients, we randomly sample 2,000 data points from $\mathcal{N}(0, 1)$.

B. Convergence Results of Randomized Advantage Transformation

Consider the regularized least-squares objective

$$\min_{\mathbf{g} \in \mathbb{R}^p} \frac{1}{2} \|\mathbf{y} - \mathbf{H}\mathbf{g}\|_2^2 + \frac{\lambda}{2} \|\mathbf{g}\|_2^2, \quad (31)$$

with unique solution

$$\mathbf{g}^* := (\mathbf{H}^\top \mathbf{H} + \lambda \mathbf{I})^{-1} \mathbf{H}^\top \mathbf{y}. \quad (32)$$

At iteration j , RAT samples a minibatch τ_j and performs the update

$$\mathbf{g}_{j+1} = \mathbf{g}_j + \mathbf{H}_{\tau_j}^\top (\lambda \mathbf{I} + \mathbf{H}_{\tau_j} \mathbf{H}_{\tau_j}^\top)^{-1} (\mathbf{y}_{\tau_j} - \mathbf{H}_{\tau_j} \mathbf{g}_j). \quad (33)$$

Define

$$\mathbf{P}_\tau := \mathbf{H}_\tau^\top (\lambda \mathbf{I} + \mathbf{H}_\tau \mathbf{H}_\tau^\top)^{-1} \mathbf{H}_\tau, \quad \mathbf{S}_\tau := \mathbf{H}_\tau^\top (\lambda \mathbf{I} + \mathbf{H}_\tau \mathbf{H}_\tau^\top)^{-1}. \quad (34)$$

Let the estimation error be

$$\mathbf{e}_j := \mathbf{g}_j - \mathbf{g}^*. \quad (35)$$

Lemma 2. For any minibatch τ , the matrix $\mathbf{P}_\tau$ satisfies

$$0 \preceq \mathbf{P}_\tau \preceq \mathbf{I}, \quad \mathbf{P}_\tau^2 \preceq \mathbf{P}_\tau.$$

Proof. Let $\mathbf{H}_\tau = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$ be the singular value decomposition. Then

$$\mathbf{P}_\tau = \mathbf{V}\mathbf{\Sigma}^\top (\lambda \mathbf{I} + \mathbf{\Sigma}\mathbf{\Sigma}^\top)^{-1} \mathbf{\Sigma}\mathbf{V}^\top = \mathbf{V} \text{diag}\left(\frac{\sigma_i^2}{\lambda + \sigma_i^2}\right) \mathbf{V}^\top.$$

Each eigenvalue lies in $[0, 1)$, implying $0 \preceq \mathbf{P}_\tau \preceq \mathbf{I}$. Since $x^2 \leq x$ for $x \in [0, 1]$, we also have $\mathbf{P}_\tau^2 \preceq \mathbf{P}_\tau$. $\square$

Lemma 3. Under the assumptions of full column rank of $\mathbf{H}$ and full data coverage,

$$\mu := \lambda_{\min}(\mathbb{E}[\mathbf{P}_\tau]) > 0.$$

Proof. Recall

$$\mathbf{P}_\tau = \mathbf{H}_\tau^\top (\lambda \mathbf{I} + \mathbf{H}_\tau \mathbf{H}_\tau^\top)^{-1} \mathbf{H}_\tau \succeq 0, \quad \text{and} \quad \mathbf{v}^\top \mathbf{P}_\tau \mathbf{v} = 0 \iff \mathbf{H}_\tau \mathbf{v} = 0.$$

We prove $\mathbb{E}[\mathbf{P}_\tau] \succ 0$ by showing that no nonzero vector $\mathbf{v}$ can satisfy $\mathbf{H}_\tau \mathbf{v} = 0$ almost surely. Fix any $\mathbf{v} \neq 0$. Since $\mathbf{H}$ has full column rank, we have $\mathbf{H}\mathbf{v} \neq 0$, hence there exists at least one index i such that $\mathbf{h}_i^\top \mathbf{v} \neq 0$. By full data coverage, $\mathbb{P}(i \in \tau) > 0$. On the event $\{i \in \tau\}$, the submatrix $\mathbf{H}_\tau$ contains row $\mathbf{h}_i^\top$, and thus $\mathbf{H}_\tau \mathbf{v} \neq 0$ (because its i -th component equals $\mathbf{h}_i^\top \mathbf{v} \neq 0$). Therefore,

$$\mathbb{P}(\mathbf{H}_\tau \mathbf{v} \neq 0) \geq \mathbb{P}(i \in \tau) > 0.$$

Consequently, $\mathbf{H}_\tau \mathbf{v} = 0$ cannot hold almost surely for any nonzero $\mathbf{v}$.

Since $(\lambda \mathbf{I} + \mathbf{H}_\tau \mathbf{H}_\tau^\top)^{-1} \succ 0$, we have $\mathbf{v}^\top \mathbf{P}_\tau \mathbf{v} > 0$ whenever $\mathbf{H}_\tau \mathbf{v} \neq 0$, and hence

$$\mathbf{v}^\top \mathbb{E}[\mathbf{P}_\tau] \mathbf{v} = \mathbb{E}[\mathbf{v}^\top \mathbf{P}_\tau \mathbf{v}] > 0 \quad \text{for all } \mathbf{v} \neq 0.$$

Thus $\mathbb{E}[\mathbf{P}_\tau] \succ 0$, which implies $\mu = \lambda_{\min}(\mathbb{E}[\mathbf{P}_\tau]) > 0$. $\square$

We first analyze the idealized case where minibatch targets are exact. For all minibatches τ , $\mathbf{y}_\tau = \mathbf{H}_\tau \mathbf{g}^*$.

Theorem 3 (Linear convergence of RAT). *Assume minibatches τ_j are sampled i.i.d. from an arbitrary distribution. Define $\mu := \lambda_{\min}(\mathbb{E}[\mathbf{P}_\tau])$, then*

$$\mathbb{E}\|\mathbf{g}_j - \mathbf{g}^*\|_2^2 \leq (1 - \mu)^j \|\mathbf{g}_0 - \mathbf{g}^*\|_2^2. \quad (36)$$

Proof. Using the noise-free assumption, the update Equation (33) becomes

$$\mathbf{g}_{j+1} = \mathbf{g}_j - \mathbf{P}_{\tau_j}(\mathbf{g}_j - \mathbf{g}^*).$$

Subtracting $\mathbf{g}^*$ yields

$$\mathbf{e}_{j+1} = (\mathbf{I} - \mathbf{P}_{\tau_j})\mathbf{e}_j.$$

Conditioned on $\mathbf{e}_j$ and τ_j ,

$$\|\mathbf{e}_{j+1}\|_2^2 = \mathbf{e}_j^\top (\mathbf{I} - 2\mathbf{P}_{\tau_j} + \mathbf{P}_{\tau_j}^2) \mathbf{e}_j.$$

Since $\mathbf{P}_{\tau_j}^2 \preceq \mathbf{P}_{\tau_j}$,

$$\mathbf{I} - 2\mathbf{P}_{\tau_j} + \mathbf{P}_{\tau_j}^2 \preceq \mathbf{I} - \mathbf{P}_{\tau_j},$$

and therefore

$$\|\mathbf{e}_{j+1}\|_2^2 \leq \|\mathbf{e}_j\|_2^2 - \mathbf{e}_j^\top \mathbf{P}_{\tau_j} \mathbf{e}_j.$$

Taking conditional expectation,

$$\mathbb{E}[\|\mathbf{e}_{j+1}\|_2^2 \mid \mathbf{e}_j] \leq \|\mathbf{e}_j\|_2^2 - \mathbf{e}_j^\top \mathbb{E}[\mathbf{P}_\tau] \mathbf{e}_j.$$

Since $\mathbf{e}_j^\top \mathbb{E}[\mathbf{P}_\tau] \mathbf{e}_j \geq \mu \|\mathbf{e}_j\|_2^2$,

$$\mathbb{E}[\|\mathbf{e}_{j+1}\|_2^2 \mid \mathbf{e}_j] \leq (1 - \mu) \|\mathbf{e}_j\|_2^2.$$

Iterating proves the claim. $\square$

We now consider stochastic targets, as in reinforcement learning. For each minibatch τ , $\mathbf{y}_\tau = \mathbf{H}_\tau \mathbf{g}^* + \boldsymbol{\xi}_\tau$, where the noise $\boldsymbol{\xi}_\tau$ satisfies $\mathbb{E}[\boldsymbol{\xi}_\tau \mid \tau] = \mathbf{0}$.

Theorem 4 (Convergence with error floor). *Define $\eta^2 := \mathbb{E}[\|\mathbf{S}_\tau \boldsymbol{\xi}_\tau\|_2^2]$, then*

$$\mathbb{E}\|\mathbf{g}_j - \mathbf{g}^*\|_2^2 \leq (1 - \mu)^j \|\mathbf{g}_0 - \mathbf{g}^*\|_2^2 + \frac{\eta^2}{\mu}. \quad (37)$$

Proof. Substituting the stochastic model into Equation (33) yields

$$\mathbf{e}_{j+1} = (\mathbf{I} - \mathbf{P}_{\tau_j})\mathbf{e}_j + \mathbf{S}_{\tau_j} \boldsymbol{\xi}_{\tau_j}.$$

Squaring and taking conditional expectation, the cross term vanishes since $\mathbb{E}[\boldsymbol{\xi}_{\tau_j} \mid \tau_j] = \mathbf{0}$, giving

$$\mathbb{E}[\|\mathbf{e}_{j+1}\|_2^2 \mid \mathbf{e}_j] \leq (1 - \mu) \|\mathbf{e}_j\|_2^2 + \eta^2.$$

Unrolling the resulting recursion completes the proof. $\square$

C. Implementation Details

We implemented RAT using the per-sample gradients feature in PyTorch³. Specifically, we first compute the per-sample gradients of the policy network's outputs with respect to its parameters using the built-in function `torch.func.grad`, `torch.func.vmap` and `torch.func.functional_call`, and then flat these per-sample gradients to compute $\mathbf{H}$. When computing $\mathbf{H}\mathbf{H}^\top$, we average over the samples in the mini-batch. We use `torch.linalg.solve` to solve the linear system involving the damped Fisher matrix $(\lambda \mathbf{I} + \mathbf{H}\mathbf{H}^\top)$, and thus apply the advantage transformation. This is a faster and more numerically stable way than performing the computations separately.

Besides, we also incorporated the following training techniques:

³https://docs.pytorch.org/tutorials/intermediate/per_sample_grads.html

Observation normalization. We normalize the observations with running mean and standard deviation as in (Schulman et al., 2017b) for all the MuJoCo tasks and set the clip range to $[-5, 5]$. For tasks with image observations, we normalize the pixel values to $[0, 1]$ by dividing them by 255 and then normalize each pixel value with 0.5 mean and 0.5 standard deviation (to ensure the pixel values are in the range of $[-1, 1]$). We also stack the last three frames as the input to the policy network. We found that observation normalization is crucial for stabilizing training, especially for tasks in the Mujoco suite. Note that after observation normalization, we re-evaluate the action distribution’s mean and variance, i.e., $\mu(s)$ and $\sigma(s)$, to ensure that the action distribution is consistent with the normalized observations.

Advantage normalization. We use the Generalized Advantage Estimation (GAE) (Schulman et al., 2016) to compute the advantage estimates. We then normalize the advantage estimates to have zero mean and unit standard deviation within each batch as in (Schulman et al., 2017b). We found that advantage normalization is important for stabilizing training, especially when using high learning rates.

PopArt value normalization. We use the PopArt normalization technique (Hessel et al., 2019) to normalize the value function targets. Specifically, we maintain running estimates of the mean μ and standard deviation σ of the value function targets and normalize the targets as $(G_t - \mu)/\sigma$. We also adjust the parameters of the value network to account for the change in normalization following the procedure described in (Hessel et al., 2019). We set the decay rate for the running estimates to 0.99999. One slight improvement we made in our implementation is that we correct the bias in the running estimates of the mean and standard deviation by dividing the estimates by $(1 - \text{decay}^t)$ at time step t , similar to Adam (Kingma & Ba, 2015).

Gradient clipping. We clip the gradient norm to be at most 0.5 when updating the shared policy and value networks. If the gradient norm exceeds this threshold, we scale down the gradient to have a norm of 0.5. We also tried the Fisher norm clipping technique proposed in (Ba et al., 2017), but found that l_2 norm clipping works equally well in our experiments. When the actor and critic networks are separate, we apply gradient clipping to the policy network with a threshold of 0.5, and to the value network with a threshold of 5.0.

Action squashing. For environments with bounded action spaces, we apply a squashing function ($\tanh$) to the actions sampled from the Gaussian policy to ensure that the actions lie within the valid range. Different from the procedure described in (Haarnoja et al., 2018), we do not adjust the log-probability of the actions to account for the squashing transformation. This is because the policy ratios, KL divergences, and Fisher matrix are all invariant to such transformations, as long as the transformation is differentiable and invertible.

Ratio clamping. When computing the policy ratios, we clamp ratios to be within $[10^{-1}, 10^1]$ to avoid numerical instability.

C.1. RAT in Shared Actor-Critic

In Actor-Critic methods, the actor and critic often share a common neural architecture (Mnih et al., 2016; Wu et al., 2017). When parameters are shared, we follow Wu et al. (2017) and estimate the joint natural policy gradients for the actor and critic. Specifically, we model the value output as a Gaussian distribution with fixed variance σ^2 , i.e., $p(v|s) = \mathcal{N}(v; V(s), \sigma^2)$, where v denotes a target value obtained from Monte-Carlo rollouts or Temporal Difference (TD) methods (Sutton & Barto, 2018). The critic is trained by maximizing the log-likelihood of this distribution, and the Fisher matrix for the critic is defined with respect to the corresponding log-likelihood. In practice we set σ to 1 without loss of generality, yielding $\log p(v|s) \propto -\|v - V(s)\|^2$.

$$\max \mathbb{E}_{v \sim q} [\log p(v|s)] = \mathbb{E}_{v \sim q} \left[-\|v - V(s)\|^2 \right]$$

Under this formulation, the score function for the joint distribution $p(a, v|s)$ factorizes as

$$\nabla_{\theta} \log p(a, v|s, \theta) = \nabla_{\theta} \log \pi(a|s; \theta) + \nabla_{\theta} \log p(v|s; \theta),$$

which is used to construct the matrix H in RAT. Following Wu et al. (2017), we sample the network outputs independently for the actor and critic, and inject unit-variance Gaussian noise to the value outputs.

To apply RAT to the critic, we introduce a pseudo advantage for the value loss, such as an all-ones vector $\mathbf{1}$ with the same size as the mini-batch. Applying RAT to this pseudo advantage yields $\tilde{w}(s)$ for each state, which can be interpreted as the

natural gradient update direction for the critic. The resulting joint loss for optimizing shared actor-critic networks is

$$\mathcal{L}(\boldsymbol{\theta}) = -\mathbb{E} \left[\frac{\pi(a|s; \boldsymbol{\theta})}{\pi_{\text{old}}(a|s)} \tilde{A}(s, a) \right] + \mathbb{E} \left[\tilde{w}(s) \|v - V(s; \boldsymbol{\theta})\|^2 \right]$$

where $\tilde{A}(s, a)$ is the transformed advantage for the actor, and $\tilde{w}(s)$ is the transformed pseudo advantage for the critic. At each iteration, RAT is applied jointly to transform both the actor advantage for the critic pseudo advantage, enabling a unified and stable natural-gradient update for shared actor-critic networks.

Algorithm 1 Randomized Advantage Transformation (RAT)

1: **Input:** Initial policy parameters $\boldsymbol{\theta}_0$, batch size B , total iterations K

2: **for** $k = 0$ to $K - 1$ **do**

3: Collect on-policy samples: $\pi_{\text{old}}(a|s) \leftarrow \pi(a|s; \boldsymbol{\theta}_k)$

4: Randomly partition $\mathcal{D}_k$ into batches of size B

5: **for** each batch τ_j in $\mathcal{D}_k$ **do**

6: Form $\mathbf{H}_{\tau_j}$ using samples in τ_j

7: Apply RAT:

$$\tilde{A}_j(s, a) = \left[(\lambda \mathbf{I} + \mathbf{H}_{\tau_j} \mathbf{H}_{\tau_j}^\top)^{-1} (\mathbf{y}_{\tau_j} - \mathbf{H}_{\tau_j} \mathbf{g}_{j-1}) \right]_{(s,a)}$$

8: Estimate gradient via a single backpropagation of the following loss:

$$\mathbf{g}_j = -\frac{\partial}{\partial \boldsymbol{\theta}} \mathbb{E} \left[\frac{\pi(a|s; \boldsymbol{\theta})}{\pi_{\text{old}}(a|s)} \tilde{A}_j(s, a) \right]$$

9: Apply gradient clipping $\alpha_k := \min \left(\eta, \frac{\nu}{\|\mathbf{g}_j\|_2} \right)$.

10: Update policy parameters: $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha_k \mathbf{g}_j$

11: **end for**

12: **end for**

13: **Output:** Policy parameters $\boldsymbol{\theta}$

C.2. Gap between Theoretical Analysis and Practical Implementation

The theorems analyze RAT as a fixed-policy linear system solver, while [Algorithm 1](#) interleaves these updates with policy optimization, resulting in a time-varying sequence of systems. This makes the linear system non-stationary across inner iterations. The resulting algorithm is closer to “multiple PPO-like updates per rollout with curvature-corrected advantages” than to an iterative linear solver converging to a fixed target. To clarify the gap formally, let $\mathbf{g}_t^*$ denote the solution of the regularized least-squares problem defined by the current policy $\boldsymbol{\theta}_t$, and define the tracking error

$$\mathbf{e}_t := \|\mathbf{g}_t - \mathbf{g}_t^*\|. \quad (38)$$

At iteration t , RAT performs an update yielding $\mathbf{g}_{t+1}$. The fixed-system analysis ([Theorem 1](#)) implies a contraction:

$$\|\mathbf{g}_{t+1} - \mathbf{g}_t^*\| \leq \rho \|\mathbf{g}_t - \mathbf{g}_t^*\| = \rho \mathbf{e}_t, \quad (39)$$

where $\rho = 1 - \mu < 1$. After the policy update $\boldsymbol{\theta}_t \mapsto \boldsymbol{\theta}_{t+1}$, the target solution shifts. Under standard smoothness assumptions on $\mathbf{H}$ and $\mathbf{y}$, the solution map $\boldsymbol{\theta} \mapsto \mathbf{g}^*(\boldsymbol{\theta})$ is Lipschitz:

$$\|\mathbf{g}^*(\boldsymbol{\theta}_{t+1}) - \mathbf{g}^*(\boldsymbol{\theta}_t)\| \leq L \|\boldsymbol{\theta}_{t+1} - \boldsymbol{\theta}_t\|. \quad (40)$$

Combining these yields:

$$\mathbf{e}_{t+1} \leq \rho \mathbf{e}_t + L \|\boldsymbol{\theta}_{t+1} - \boldsymbol{\theta}_t\| \quad (41)$$

Unrolling:

$$\mathbf{e}_t \leq \rho^t \mathbf{e}_0 + L \sum_{s=0}^{t-1} \rho^{t-1-s} \|\boldsymbol{\theta}_{s+1} - \boldsymbol{\theta}_s\| \quad (42)$$

Table 4. Hyperparameters for RAT.

Hyperparameter	Value
pi lr	0.05
vf lr	0.001
lr for shared network	0.1
damping for MLP	0.1
damping for CNN & ResNet	0.5

This shows that RAT can be interpreted as a *contractive solver tracking a slowly varying sequence of systems*. Under our settings (small learning rates and gradient clipping), the drift term remains small, yielding a bounded steady-state error of order

$$\mathcal{O}(\max_t \|\theta_{t+1} - \theta_t\|) \quad (43)$$

This is a standard tracking bound for contractive iterative methods applied to slowly varying systems and provides a principled justification for the interleaved algorithm.

Specifically, while the Lipschitz constant L is not directly measurable, the step size $\|\theta_{t+1} - \theta_t\|$ is explicitly controlled by the learning rate and gradient clipping. In particular, we use ℓ_2 gradient norm clipping at 0.5 together with a learning rate 0.05 for the policy network, which ensures that each parameter update is bounded by at most $0.5 \times 0.05 = 0.025$ in ℓ_2 norm. This keeps the drift term small throughout training up to the unknown constant L . More broadly, this is precisely why gradient clipping and small learning rates are important in our implementation: they ensure that the target system evolves slowly enough for the tracking interpretation to be meaningful.

D. Hyperparameters

We summarize the hyperparameters used in our experiments in Table 3.

Table 3. Common hyperparameters used in our experiments.

Hyperparameter	Value
Discount factor γ	0.99
GAE parameter λ	0.95
Damping factor λ	10^{-1}
Mini-batch size	1024
PPO clipping parameter ϵ	0.2
Number of epochs per update	8
Number of steps per update	$256 * 32$
Gradient clipping threshold (policy)	0.5
Gradient clipping threshold (value)	5.0
PopArt decay rate	0.99999
Observation normalization clip range	$[-5, 5]$
Entropy coefficient	0

Table 5. Hyperparameters for KFAC (adopted from (George et al., 2018)).

Hyperparameter	Value
lr	0.001
momentum	0.9
stat decay	0.95
damping	0.001
kl clip	0.001
weight decay	0
TCov	1
TInv	10
batch averaged	True

Table 6. Hyperparameters for PPO.

Hyperparameter	Value
pi lr	0.001
vf lr	0.001
lr for shared networks	0.001
clip range	0.2
# of epochs	4
# of mini-batches	8
# of steps per update	256 * 32

Architecture details:

- For MuJoCo tasks, we use a two-layer MLP with 256 hidden units per layer and tanh activations for both the policy and value networks.
- For Procgen tasks, we use the same ResNet architecture as in (Cobbe et al., 2020): four residual blocks with 16, 32, and 32 filters respectively, followed by a fully connected layer with 256 units. We use ReLU activations after each layer.

E. Additional Experimental Results

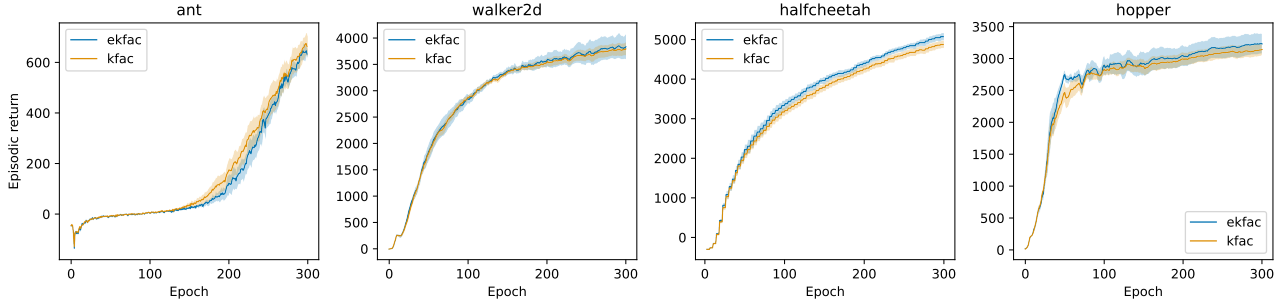


Figure 6. Comparing KFAC and EKFAC on Continuous Control Tasks. EKFAC performs similar to KFAC in most tasks.

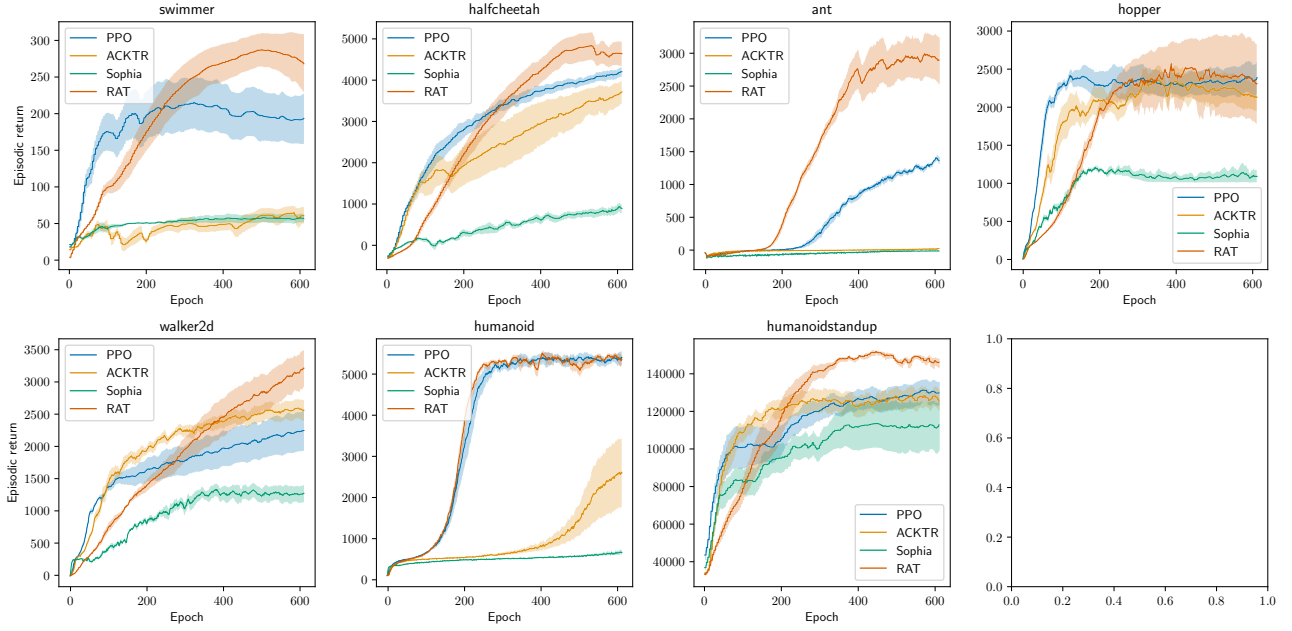


Figure 7. Optimizing MLP Policies on Continuous Control Tasks with Shared Actor-Critic Networks. RAT outperforms KFAC and FVP+CG in most tasks. The shaded region denotes the standard deviation over 5 random seeds.

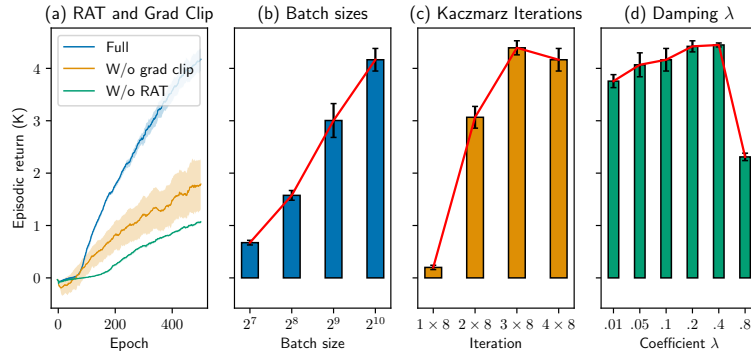


Figure 8. Ablation study and sensitivity analysis of RAT on Ant.